

Overview

口腔機能・栄養評価

配布・運用の全体像

Generated from README.md / 2026-05-04 22:12

PDF 化するときは A4 でそのまま印刷できます。見出し、図、コードブロックが崩れにくく、改ページの空きが大きくなりすぎないように調整しています。

口腔機能・栄養評価システムを Synology NAS / Docker で自己ホストし、Tailscale Serve で HTTPS 終端するための構成です。

このフォルダの [index.html](#) は Claude artifact の wrapper ですが、[server.py](#) がその中に埋め込まれた本体 HTML を抽出し、保存処理だけを SQLite API に差し替えて配信します。これにより、既存レイアウトをほぼそのまま維持しつつ、複数端末で記録を共有できます。

関連資料

- Synology + Tailscale 配置手順: [DEPLOY_SYNOLOGY_JA.html](#) / [DEPLOY_SYNOLOGY_JA.pdf](#)
- Windows 利用者向け Tailscale 接続ガイド: [TAILSCALE_CLIENT_GUIDE_JA.html](#) / [TAILSCALE_CLIENT_GUIDE_JA.pdf](#)
- タブレット利用者向け Tailscale 接続ガイド: [TAILSCALE_TABLET_GUIDE_JA.html](#) / [TAILSCALE_TABLET_GUIDE_JA.pdf](#)
- タブレット利用者向け案内文テンプレート: [TAILSCALE_TABLET_MESSAGE_TEMPLATE_JA.html](#) / [TAILSCALE_TABLET_MESSAGE_TEMPLATE_JA.pdf](#)
- タブレット利用者向け QR 案内シート: [TAILSCALE_TABLET_QR_SHEET_JA.html](#) / [TAILSCALE_TABLET_QR_SHEET_JA.pdf](#)
- 初心者向け運用マニュアル: [OPERATIONS_MANUAL_PDF_JA.html](#) / [OPERATIONS_MANUAL_JA.pdf](#)
- Windows 用かんたん起動: [TailscaleClientLauncher.cmd](#)
- 起動ツール設定: [TailscaleClientLauncher.settings.json](#)

- 実運用設定のひな形: `.env.example`
- 現在の実運用設定: Git 管理対象外の `.env` を各環境で配置して使います。

アプリ画面右上の ? ヘルプから開く README.html では、配布向けの HTML / PDF 資料も直接開けます。

配布パッケージの作り方

配布用の PDF 再生成と package フォルダ作成は、`build_package.cmd` または `build_package.ps1` でまとめて実行できます。

1. `build_package.cmd` をダブルクリックする
2. または PowerShell で `./build_package.ps1` を実行する
3. 完了後に `package/` 配下へ管理者向け、Windows 利用者向け、タブレット配布向けのフォルダと ZIP が作成される

この処理では Markdown から `README.html` と各 HTML / PDF を作り直すため、README や配布資料を更新した後は再実行してください。

共有開発の運用ルール

今後の役割は次で固定します。

1. 開発フォルダ: `\\192.168.11.200\全社共有\IT\アプリ開発\神谷\kouku-kinou`
2. 運用フォルダ: `\\192.168.11.200\docker\kouku-kinou`
3. GitHub: <https://github.com/peacesoleiljapan-star/kouku-kinou.git>

考え方:

1. コード、文書、配布資料の更新は開発フォルダだけで行います。
2. 運用フォルダは Container Manager に読ませる配備先です。原則として `.env` と `data/` 以外を直接編集しません。
3. GitHub は履歴管理と復旧元です。開発フォルダで変更したものを `commit / push` して残します。

各 PC の Python 環境:

1. `.venv` を共有フォルダの中に置かないでください。

2. 各 PC ごとにローカルの Python / venv を使います。
3. `build_package.ps1` は環境変数 `KOUKU_KINOU_PYTHON` があれば、その `python.exe` を優先して使います。
4. この PC では
`KOUKU_KINOU_PYTHON=C:\Users\user\Documents\vscrepo\.venv\Scripts\python.exe` を設定済みです。
5. 別の PC では、共有開発フォルダ直下の `setup_shared_dev_pc.cmd` を 1 回実行すれば、`git safe.directory` と `KOUKU_KINOU_PYTHON` の設定を入れられます。必要なら
`setup_shared_dev_pc.ps1 -PythonPath C:\path\to\python.exe` を使います。

GitHub への反映

このフォルダはすでに GitHub の `peacesoleiljapan-star/kouku-kinou` と接続済みです。今後は次の 2 通りで反映できます。

1. すぐ反映したいときは `sync_to_github_now.cmd` をダブルクリックする
2. 自動反映したいときは `setup_github_auto_sync.cmd` を 1 回だけ実行する

`setup_github_auto_sync.cmd` を実行すると、Windows のスタートアップへ自動起動を登録し、次を有効にします。

1. Windows のスタートアップへ自動起動ランチャーを登録する
2. ログオン中はこのフォルダの保存済み変更を監視し、しばらく更新が止まったら自動で `commit / push` する
3. 変更がなくても定期的に同期処理を実行する

初期設定では 60 秒ほど更新が止まると自動反映し、15 分ごとにも定期同期します。未保存の編集内容は GitHub へ送られないため、保存後に反映されます。

最小の作業手順:

1. 開発フォルダで `git pull`
2. ファイルを編集する
3. `build_package.ps1` で package を作る
4. 必要な配布物を運用フォルダへ反映する
5. 動作確認後に `git add -A` → `git commit -m "変更内容"` → `git push`

GitHub の最低限のコマンド:

```
git pull
git status
git add -A
git commit -m "変更内容"
git push
```

配布前チェック

1. 配布先に `.env` がなければ、`.env.example` をコピーして `.env` を作成します。
2. `KOUKU_KINOU_PASSWORD` を本番用の強い値へ変更します。
3. `TailscaleClientLauncher.settings.json` の `organizationName`、`appUrl`、`supportText`、`supportContact` を配布先に合わせて更新します。
4. Windows 利用者へは launcher 3 ファイルと `TAILSCALE_CLIENT_GUIDE_JA.pdf`、タブレット利用者へは URL と `TAILSCALE_TABLET_GUIDE_JA.pdf` / `TAILSCALE_TABLET_QR_SHEET_JA.pdf` を渡します。
5. 配布前に `/api/health`、保存、検索、設定画面、アプリ内ヘルプの HTML / PDF / 画像表示を確認します。

想定構成

1. アプリ本体は Synology Container Manager 上の Docker コンテナで動かします。
2. `compose.yaml` はホストの `127.0.0.1:8010` にだけ公開し、LAN から直接見えないようにします。
3. Synology 上の Tailscale Serve が `https://diskstation.tail632bc4.ts.net/` を終端し、ローカルの `http://127.0.0.1:8010` へ流します。
4. 利用者は Tailscale に接続したうえで、その HTTPS URL からアクセスします。

DSM のリバースプロキシや独自ドメイン、ポート開放は前提にしません。

認証とアクセス制御

既定では認証を有効にして起動します。Tailscale 運用向けの既定値は `tailscale-or-password` です。

主な環境変数:

- `KOUKU_KINOU_AUTH_MODE` : `password` / `tailscale` / `tailscale-or-password`
- `KOUKU_KINOU_PASSWORD` : 管理者用パスワード。`tailscale-or-password` の予備入口として使います。
- `KOUKU_KINOU_SESSION_TTL_MINUTES` : パスワードログイン時のセッション有効時間。既定 480 分
- `KOUKU_KINOU_SECURE_COOKIE` : `1` で Secure Cookie を有効化。Tailscale HTTPS では `1` を維持します。
- `KOUKU_KINOU_ALLOWED_NETWORKS` : 追加の IP 制限が必要な場合だけ使います。
- `KOUKU_KINOU_TRUST_PROXY` : `X-Forwarded-For` ベースの制御が必要なときだけ `1` にします。

`tailscale-or-password` では、Tailscale の HTTPS URL から来た利用者はヘッダーで自動認証されます。Tailscale が使えない場合だけ、管理者用パスワードでの入口が残ります。

ローカル起動

`.env` がある場合は、`python server.py` 実行時にも自動で読み込みます。

PowerShell 例:

```
python server.py --auth-mode password --host 127.0.0.1 --port 8010
```

認証を切って画面確認だけしたい場合は以下です。

```
python server.py --no-auth --host 127.0.0.1 --port 8010
```

起動後に以下へアクセスします。

- `http://localhost:8010/`
- ヘルプ: `http://localhost:8010/readme.html`
- ヘルスチェック: `http://localhost:8010/api/health`

SQLite の保存先は既定で `data/records.db` です。環境変数 `KOUKU_KINOU_DB` で変更できます。

保存ルール:

- 利用者は `氏名 + 生年月日` で識別します

- 同じ利用者の同じ評価日は新規追加ではなく上書き更新します
- 同じ利用者でも評価日が違えば履歴として別記録を残します
- 記録一覧では氏名、ふりがな、生年月日、評価日で検索できます

確認用サンプル投入:

```
python seed_sample_records.py
```

既存データを消して入れ直す場合は `python seed_sample_records.py --replace` を使います。

Docker / Synology

配布先に `.env` がなければ `.env.example` をコピーして `.env` を作成します。すでに `.env` がある場合も、まず中身を確認し、必要なら管理者用パスワードを変更してください。

```
KOUKU_KINOU_AUTH_MODE=tailscale-or-password
KOUKU_KINOU_PASSWORD=change-this-admin-password
KOUKU_KINOU_SESSION_TTL_MINUTES=480
KOUKU_KINOU_SECURE_COOKIE=1
KOUKU_KINOU_DATA_DIR=./data
KOUKU_KINOU_ALLOWED_NETWORKS=
KOUKU_KINOU_TRUST_PROXY=0
```

Synology で `unable to open database file` が出る場合は、`KOUKU_KINOU_DATA_DIR` を `./data` から NAS 上の実フォルダへ明示してください。例: `/volume1/docker/kouku-kinou/data`

コンテナ起動:

```
docker compose up --build -d
docker compose ps
```

`docker compose ps` の `STATUS` で `healthy` になれば、`/api/health` まで含めて通っています。

`/api/health` が `{"status": "error", "error": "unable to open database file"}` を返す場合は次を順に確認します。

1. `.env` の `KOUKU_KINOU_DATA_DIR` が意図したフォルダを指している
2. そのフォルダが Synology 上に実在する
3. Container Manager でそのフォルダを読書きできる

4. 再ビルド後に `records.db`、`records.db-wal`、`records.db-shm` が作成される

現在の公開 URL は <https://diskstation.tail632bc4.ts.net/> です。

Synology では DSM で SSH を有効にしたうえで NAS に入り、root 権限で Tailscale Serve を設定します。

Tailscale Serve 設定:

```
tailscale serve --bg 8010
tailscale serve status
```

`tailscale` コマンドが見つからない場合は、Synology のパッケージ実体を直接呼びます。

```
/var/packages/Tailscale/target/bin/tailscale serve --bg 8010
/var/packages/Tailscale/target/bin/tailscale serve status
```

`tailscale serve --bg 8010` は、Tailscale の HTTPS URL から <http://127.0.0.1:8010> へ転送します。初回は HTTPS 有効化の承認画面が開くことがあります。

利用者向け配布物

Windows 利用者には、次の 3 ファイルをまとめて配布できます。

1. `TailscaleClientLauncher.cmd`
2. `TailscaleClientLauncher.ps1`
3. `TailscaleClientLauncher.settings.json`

配布用の `TailscaleClientLauncher.settings.json` には

<https://diskstation.tail632bc4.ts.net/> を設定済みです。利用者はコマンド入力なしで Tailscale 導入とアプリ起動を進められます。

配布前に `organizationName`、`appUrl`、`supportText`、`supportContact` を現場向けに更新してください。特に `supportContact` は利用者が困ったときの連絡先になるため、空欄のまま配らないほうが安全です。

Windows 利用者へ操作手順も渡す場合は `TAILSCALE_CLIENT_GUIDE_JA.pdf` を同梱すると、そのまま案内に使えます。

タブレット利用者には、ファイル配布より次の 2 つだけを案内するほうが簡単です。

1. Tailscale の導入とサインイン方法

2. アプリ URL <https://diskstation.tail632bc4.ts.net/>

タブレット向けの詳しい案内は [TAILSCALE_TABLET_GUIDE_JA.html](#) にまとめています。配布時にそのまま使う文面は [TAILSCALE_TABLET_MESSAGE_TEMPLATE_JA.html](#)、紙配布用の QR 案内は [TAILSCALE_TABLET_QR_SHEET_JA.html](#) を使えます。

配布用の見やすい版としては [TAILSCALE_TABLET_GUIDE_JA.pdf](#)、[TAILSCALE_TABLET_MESSAGE_TEMPLATE_JA.pdf](#)、[TAILSCALE_TABLET_QR_SHEET_JA.pdf](#) も同梱しています。

実装メモ

- 画面配信時に `localStorage` を使う元コードを API 呼び出しへ変換しています。
- 記録は SQLite に JSON として保存されます。
- 利用者識別は `氏名 + 生年月日`、評価識別は `氏名 + 生年月日 + 評価日` です。
- 同一利用者・同一評価日の保存は上書き、別日の評価は履歴追加です。
- 記録一覧には検索欄があり、氏名、ふりがな、生年月日、評価日で絞り込みできます。
- 共有対象は同じ NAS 上の単一 DB なので、PC とタブレットで同じ記録一覧を参照できます。
- `compose.yaml` は loopback のみに bind するため、Tailscale Serve を通らない直接アクセスを減らせます。
- `compose.yaml` の healthcheck は `/api/health` で DB と画面テンプレートの準備完了を確認します。
- `tailscale` または `tailscale-or-password` では、Tailscale identity header を使った認証に対応しています。
- [README.html](#) から PDF 資料と `assets/` 配下の画像付きマニュアルを開けるようにしているため、配布後もブラウザーだけで資料確認を完結できます。
- `index.html` 自体は元 artifact のソースとして残しています。

Source: README.md / Generated by build_docs.py